

西安交通大学

课程实验报告

实验名称：文本处理算法实践

组 长：林圣翔

组 员：易怀川、李彦铨

报告日期：2025.11.28



一、实验目的

- 系统掌握经典文本处理算法的核心原理与实现。
- 培养模型构建、调优与评估的完整工程实践技能。
- 提升针对现实场景的建模能力与解决方案设计水平。

二、实验内容

- 针对 BPE、LSTM、Transformer 三个关键算法，在不依赖高级机器学习库的前提下，完成从零开始的手动实现。
- 通过调整关键参数，系统分析不同配置对模型训练效果及资源占用的影响。
- 对代码进行工程化重构，强调模块化与接口设计，确保系统具备良好的可扩展性。

三、实验环境

- 系统环境：基于 Ubuntu 20.04.6 LTS 版本
- 编程语言：Python 3.13

四、数据集简介

本次实验采用的数据集为 TinyStoriesV2-GPT4，该数据集发布在 Hugging Face 平台上。训练集与验证集分别通过以下链接获取。

数据集	URL
训练集	https://huggingface.co/datasets/roneneldan/TinyStories/resolve/main/TinyStoriesV2-GPT4-train.txt
测试集	https://huggingface.co/datasets/roneneldan/TinyStories/resolve/main/TinyStoriesV2-GPT4-valid.txt

该数据集是一个独特的大规模文本数据集，其核心特点在于使用受限的词汇和简单的语法结构来讲述完整的叙事故事。该数据集的创建旨在探究语言模型在理解与生成连贯文本时，是否能够不依赖于复杂的语言现象。本实验采用的 V2-GPT4 版本由强大的大语言模型 GPT-4 生成，相较于早期版本，在故事的连贯性、趣味性和语言质量上均有显著提升。

五、算法原理

1. BPE 算法

BPE (Byte Pair Encoding) 是一种通过迭代合并高频字符对来构建子词词

表的算法，在自然语言处理的分词器设计中应用广泛。其核心思想是从字符级单位开始，逐步合并出现频率最高的相邻字符对，形成更长的子词单元，直至词表规模达到预设上限。

在本实验中，我们未直接使用现有的分词库（如 Hugging Face tokenizers），而是从底层自主实现了完整的 BPE 算法。该实现涵盖训练与分词两个主要阶段：训练阶段从 256 个单字节基础词汇出发，统计预分词后文本中的字节对共现频率，迭代合并最高频的字节对以生成新词元，直至词表达到目标大小；分词阶段采用双路径处理机制，一方面基于前缀树快速识别并保留特殊标记的完整性，另一方面将常规文本按预定义模式分块后应用训练所得的 BPE 合并规则进行子词划分，最终将词元映射为对应的词汇 ID。整个实现依托优化的数据结构设计，在确保功能完备性的同时，也有效提升了分词的执行效率。

2. LSTM

LSTM 主要用于文本分类等自然语言处理任务。模型首先通过嵌入层（Embedding）将输入的单词索引转换为稠密的词向量表示，然后利用 LSTM 层捕捉文本序列中的长期依赖关系，通过其门控机制（遗忘门、输入门、输出门）有效学习上下文特征。在模型最后，仅提取 LSTM 输出序列的最后一个时间步的隐藏状态（代表整个序列的语义信息），并通过全连接层映射到目标类别空间，输出分类结果。该模型能够处理变长文本序列，具有结构简单、参数量适中的特点，是自然语言处理领域的基础架构之一。

在本实验中，所实现的 LSTM 模型围绕语言建模任务构建，其核心配置包含四个关键参数：`d_model` 统一规定词向量维度与隐藏状态大小，`num_layers` 控制网络深度，`vocab_size` 与 `context_length` 则分别设定词汇表规模与输入序列最大长度。模型首先通过嵌入层将词索引映射为向量表示，经多层 LSTM 捕捉序列依赖，最终通过包含 Dropout 正则化的多层前馈网络输出词汇概率分布。该设计在保持结构简洁的同时，支持基于状态传递的序列生成任务，为语言建模提供了一个高效的基准实现。

3. Transformer

Transformer 主要用于自然语言处理任务。模型首先通过嵌入层将输入的单词索引转换为 `d_model` 维的稠密向量表示，然后使用多头注意力机制来捕捉输入序列中各个位置之间的全局依赖关系，其中 `num_heads` 参数控制注意力头的数量。在前向传播过程中，模型对输入序列进行自注意力计算后，通过对所有位置的输出取平均来获得整个序列的表示，最后通过全连接层将其映射到目标类别空间输出分类结果。该模型摒弃了传统的循环神经网络结构，完全依赖自注意力机制来建模序列关系，具有并行计算效率高、能捕获长距离依赖等优势，是当前自然语言处理领域的主流架构之一。

本实验所实现的 Transformer 模型，主要参考 Stanford CS336 课程（2025 年春季）所介绍的架构形式，并在其基础上进行了精简与优化，包括对部分模块进行删减或调整，以及对特定函数进行性能优化。此外，在实现过程中，为深入理解模型细节，我们未直接使用 PyTorch 框架中的高级封装类，而是从底层开始手动实现了所有核心组件。

六、实验过程

1. 环境配置

1.1. 创建 Conda 虚拟环境

```
conda create -n DM python=3.13
```

1.2. 激活 Conda 虚拟环境

```
conda activate DM
```

1.3. 安装项目依赖包

```
pip install -r requirements.txt
```

2. 代码实现

相关代码文件及功能如下

```
root/
├── README.md
├── requirements.txt          # 项目依赖与环境配置
├── lstm_scripts/           # LSTM 模型相关脚本
│   ├── config.json         # LSTM 模型训练与生成的配置文件
│   ├── generate.py          # 使用训练好的 LSTM 模型生成文本
│   ├── model.py             # LSTM 模型结构定义
│   ├── train.py             # LSTM 模型训练流程
│   └── utils.py             # LSTM 相关辅助函数
├── transformer_scripts/    # Transformer 模型相关脚本
│   ├── config.json         # Transformer 模型配置文件
│   ├── generate.py          # Transformer 模型文本生成
│   ├── model.py             # Transformer 模型结构定义
│   ├── train.py             # Transformer 模型训练流程
│   └── utils.py             # Transformer 相关辅助函数
├── scripts/                # 数据预处理与 tokenizer 训练脚本
│   ├── tokenize_data.py     # 数据 tokenization 处理
│   └── train_bpe.py          # 训练 BPE tokenizer
├── tests/                  # 测试代码（参考 Stanford CS336 2025 春季课程 Assignment 1）
│   ├── adapters.py          # Transformer 底层函数手写实现（如注意力、前馈网络等）
│   └── ...                  # 其他文件
```

2.1. 分词器

2.1.1. 初始化词汇表

```
vocab: Dict[int, bytes] = {i: bytes([i]) for i in range(256)}
next_id = 256
special_token_bytes = [token.encode("utf-8") for token in special_tokens]
for token_bytes in special_token_bytes:
    if token_bytes not in vocab.values():
        vocab[next_id] = token_bytes
        next_id += 1
```

2. 1. 2. 预分词处理

使用正则表达式将文本分割为可处理的单元。

```
PAT = r'""s|t|re|ve|m|ll|d| ?[p{L}]+| ?[p{N}]+| ?[^s\p{L}\p{N}]+\s+(?!S)\s+'''
```

处理流程：按特殊 token 分割文本，对每个普通文本块进行正则匹配，将匹配到的单词转换为字节序列并统计频率。

2. 1. 3. BPE 合并算法核心

```
while len(vocab) < vocab_size:
    # 统计所有相邻 pair 的频率
    pair_counts = defaultdict(int)
    for token, cnt in pre_tokens_cnt.items():
        if len(token) <= 1:
            continue
        pairs = [(token[i], token[i+1]) for i in range(len(token)-1)]
        for pair in pairs:
            pair_counts[pair] += cnt
    # 选择最频繁的 pair 进行合并
    best_pair = max(pair_counts.items(), key=lambda x: (x[1], x[0]))[0]
    a, b = best_pair
    new_token = a + b
    # 添加到词汇表
    vocab[next_id] = new_token
    next_id += 1
    # 在训练数据中应用合并
    # ... 更新所有包含该 pair 的 token 序列
    merges.append((a, b)) # 记录合并规则
```

2. 1. 4. BPE 编码过程

(1) 特殊 token 处理

```
def encode(self, text: str) -> list[int]:
    text_bytes = text.encode('utf-8')
    special_positions = self._find_special_tokens(text_bytes)
    # 分离普通文本和特殊 token
    tokens = []
    last_pos = 0
    for start, end, special_token in special_positions:
        # 处理普通文本部分
        if start > last_pos:
            ordinary_bytes = text_bytes[last_pos:start]
            ordinary_str = ordinary_bytes.decode('utf-8', errors='replace')
            for match in self.pat.finditer(ordinary_str):
                chunk = match.group(0)
                chunk_bytes = chunk.encode('utf-8')
                sub_ids = self._bpe_encode(chunk_bytes) # BPE 编码
                tokens.extend(sub_ids)
```

```

        # 添加特殊 token
        tokens.append(self.vocab_inv[special_token])
        last_pos = end
    return tokens

```

使用字典树（_build_special_trie）高效识别特殊 token，特殊 token 不会被 BPE 拆分，始终保持为单个 token，在编码时优先识别特别 token。在 _find_special_tokens 方法中被使用。

```

def _find_special_tokens(self, text_bytes: bytes):
    positions = []
    i = 0
    while i < len(text_bytes):
        node = self.special_trie # 使用构建的字典树
        j = i
        matched_token = None
        # 在字典树中匹配最长特殊 token
        while j < len(text_bytes) and text_bytes[j] in node:
            node = node[text_bytes[j]]
            j += 1
        if None in node: # 找到完整特殊 token
            matched_token = node[None]
        if matched_token:
            positions.append((i, j, matched_token))
            i = j # 跳过已匹配的部分
        else:
            i += 1
    return positions

```

（2）BPE 编码核心

```

def _bpe_encode(self, text_bytes: bytes) -> list[int]:
    # 初始化为单字节 token
    tokens = [bytes([b]) for b in text_bytes]
    # 应用合并规则
    while len(tokens) > 1:
        best_pair = None
        best_priority = float('inf')
        # 查找可合并的 pair（按训练顺序）
        for i in range(len(tokens) - 1):
            pair = (tokens[i], tokens[i + 1])
            if pair in self.merge_priority:
                priority = self.merge_priority[pair]
                if priority < best_priority: # 数字越小，优先级越高
                    best_priority = priority
                    best_pair = pair
        if best_pair is None:

```

```

        break # 无更多合并
# 执行合并
a, b = best_pair
merged = a + b
new_tokens = []
i = 0
while i < len(tokens):
    if i + 1 < len(tokens) and tokens[i] == a and tokens[i + 1] == b:
        new_tokens.append(merged)
        i += 2
    else:
        new_tokens.append(tokens[i])
        i += 1
tokens = new_tokens
# 转换为 token ID
token_ids = []
for token in tokens:
    if token in self.vocab_inv:
        token_ids.append(self.vocab_inv[token])
    else:
        # 回退到单字节
        for byte in token:
            byte_token = bytes([byte])
            token_ids.append(self.vocab_inv[byte_token])
return token_ids

```

(3) 解码过程

```

def decode(self, token_ids: list[int]) -> str:
    bytes_list = []
    for token_id in token_ids:
        if token_id in self.vocab:
            bytes_list.append(self.vocab[token_id])
        else:
            bytes_list.append(b" ") # 处理未知 token
    combined_bytes = b"".join(bytes_list)
    return combined_bytes.decode('utf-8', errors='replace')

```

2. 2. LSTM

2. 2. 1. 基础组件

```

class Linear(nn.Module):
    def __init__(self, d_in: int, d_out: int):
        super().__init__()
        std = math.sqrt(2 / (d_in + d_out))
        self.weight: Float[Tensor, "d_out d_in"] = nn.Parameter(
            nn.init.trunc_normal_(torch.empty(d_out, d_in), std=std, a=-3*std, b=3*std),

```

```

        requires_grad=True
    )
    def forward(self, x: Float[Tensor, "... d_in"]) -> Float[Tensor, "... d_out"]:
        d_in = self.weight.shape[1]
        d_out = self.weight.shape[0]
        return run_linear(d_in, d_out, self.weight, x)

```

2. 2. 2. 核心实现

```

class LSTMNetwork(nn.Module):
    def __init__(self, vocab_size: int, context_length: int, d_model: int,
                  num_layers: int, num_heads: int = None, d_ff: int = None,
                  rope_theta: float = None):
        super().__init__()
        self.config = {
            'vocab_size': vocab_size,
            'context_length': context_length,
            'd_model': d_model,
            'num_layers': num_layers,
            'num_heads': num_heads,      # 为兼容性保留
            'd_ff': d_ff,                # 为兼容性保留
            'rope_theta': rope_theta     # 为兼容性保留
        }
        self.vocab_size = vocab_size
        self.context_length = context_length
        self.d_model = d_model
        self.num_layers = num_layers
        self.embedding = Embedding(vocab_size, d_model)
        self.lstm = nn.LSTM(
            input_size=d_model,
            hidden_size=d_model,
            num_layers=num_layers,
            dropout=0.1,
            batch_first=True
        )
        self.output_layers = nn.Sequential(
            nn.Dropout(0.2), # 输出层 dropout
            Linear(d_model, 512),
            nn.Dropout(0.2),
            Linear(512, 256),
            nn.Dropout(0.2),
            Linear(256, vocab_size)
        )
        logger.info(f'LSTM Model Parameters: {self.get_num_params() / 1e6:.2f}M')

```

2. 2. 3. 前向传播实现

```

def forward(self, x: Int[Tensor, "batch sequence_length"],
            state: tuple = None) -> Float[Tensor, "batch sequence_length vocab_size"]:
    batch_size, seq_len = x.shape
    x_emb = self.embedding(x)  # (batch, seq_len, d_model)
    if state is None:
        h0 = torch.zeros(self.num_layers, batch_size, self.d_model, device=x.device)
        c0 = torch.zeros(self.num_layers, batch_size, self.d_model, device=x.device)
        state = (h0, c0)
    lstm_out, new_state = self.lstm(x_emb, state)  # (batch, seq_len, d_model)
    logits = self.output_layers(lstm_out)  # (batch, seq_len, vocab_size)
    return logits, new_state

```

2. 2. 4. 生成机制

①增量解码：初始输入 x 为上下文 tokens，循环 max_new_tokens 次，每次预测下一个 token。当输入长度超过 context_length 时，仅保留最后 context_length 个 token。

```

original_sequence_length = x.size(-1)
for _ in range(max_new_tokens):
    x = x[:, -self.context_length :] if x.size(1) > self.context_length else x
    logits = self.forward(x)
    next_token_logits = logits[:, -1]
    temperature_scaled_next_token_logits = next_token_logits / temperature
    if top_k:
        topk_values, _ = torch.topk(temperature_scaled_next_token_logits,
                                    min(top_k, temperature_scaled_next_token_logits.size(-1))),
        threshold = topk_values[:, -1]
        topk_mask = temperature_scaled_next_token_logits < threshold
        temperature_scaled_next_token_logits.masked_fill(topk_mask, float("-inf"))
    next_token_probabilities = run_softmax(temperature_scaled_next_token_logits, dim=-1)
    next_token_id = torch.multinomial(next_token_probabilities, 1)
    if eos_token_id is not None and next_token_id.item() == eos_token_id:
        break
    x = torch.cat((x, next_token_id), dim=-1)
    new_token_ids = x[:, original_sequence_length:]
    return new_token_ids

```

②采样策略

温度缩放：

```

temperature_scaled_next_token_logits = next_token_logits / temperature

```

Top-k 采样：

```

if top_k:
    topk_values, _ = torch.topk(
        temperature_scaled_next_token_logits,
        min(top_k, temperature_scaled_next_token_logits.size(-1)),
    )

```

```
threshold = topk_values[:, -1]
topk_mask = temperature_scaled_next_token_logits < threshold
temperature_scaled_next_token_logits.masked_fill(topk_mask, float("-inf"))
```

$$masked_logits = \begin{cases} logits & \text{if } logits \geq threshold \\ -\infty & \text{otherwise} \end{cases}$$

Softmax 概率计算:

```
next_token_probabilities = run_softmax(temperature_scaled_next_token_logits, dim=-1)
```

$$probabilities = softmax(logits) = \frac{e^{logits_i}}{\sum_j e^{logits_j}}$$

多项式采样:

```
next_token_id = torch.multinomial(next_token_probabilities, 1)
```

logits 除以 temperature 控制输出锐利度。仅保留概率前 k 的 token，抑制低概率长尾。检测到 eos_token_id 时提前终止生成。

2.3. Transformer

2.3.1. 神经网络基本单元

(1) Linear 层: 采用截断正态分布初始化，避免梯度消失或爆炸。实现代码位于 model.py 的 Linear 类。

```
class Linear(nn.Module):
    def __init__(self, d_in: int, d_out: int):
        std = math.sqrt(2 / (d_in + d_out))
        self.weight = nn.Parameter(
            nn.init.trunc_normal_(torch.empty(d_out, d_in), std=std, a=-3*std, b=3*std),
            requires_grad=True
        )
    def forward(self, x):
        return run_linear(d_in, d_out, self.weight, x)
```

(2) Embedding 层: 初始化标准差设为 1.0，平衡语义表示能力与训练稳定性，同时通过 run_embedding 实现高效索引。实现代码位于 model.py 的 Embedding 类。

```
class Embedding(nn.Module):
    def __init__(self, vocab_size: int, d_model: int):
        super().__init__()
        std = 1.0
        self.weight = nn.Parameter(
            nn.init.trunc_normal_(torch.empty(vocab_size, d_model), std=std, a=-3 * std, b=3 * std),
            requires_grad=True
        )
    def forward(self, token_ids: Int[Tensor, "..."]) -> Float[Tensor, "... d_model"]:
        vocab_size = self.weight.shape[0]
        d_model = self.weight.shape[1]
        return run_embedding(vocab_size, d_model, self.weight, token_ids)
```

(3) RotaryEmbedding 层: 将位置信息编码为旋转矩阵，与词嵌入逐元素相乘。

预计算位置编码缓存，前向融合位置编码，公式如下：

$$\text{RoPE}(x, t) = \begin{cases} x_{t,2i} \cdot \cos(\theta^{2i}t) - x_{t,2i+1} \cdot \sin(\theta^{2i}t) \\ x_{t,2i+1} \cdot \cos(\theta^{2i}t) + x_{t,2i} \cdot \sin(\theta^{2i}t) \end{cases}$$

其中 θ 为可调频率参数， t 为位置索引。实现于 model.py 的 RotaryEmbedding 类。

```
class RotaryEmbedding(nn.Module):
    def __init__(self, context_length: int, dim: int, theta: float = 10000.0):
        super().__init__()
        self.context_length = context_length
        self.dim = dim
        self.theta = theta
        self.register_buffer(
            "_freq_cis_cache",
            RotaryEmbedding._init_cache(context_length, dim, theta), persistent=False
        )
    @staticmethod
    def _init_cache(context_length: int, dim: int, theta: float) -> Float[Tensor, " 2 context_length half_dim"]:
        assert dim % 2 == 0
        d = torch.arange(0, dim, 2) / dim
        freqs = theta ** -d
        t = torch.arange(context_length)
        freqs = einsum(t, freqs, "t, f -> t f")
        cos, sin = torch.cos(freqs), torch.sin(freqs)
        return torch.stack((cos, sin))
    def forward(self, x: Float[Tensor, " ... seq d"], pos_ids: Int[Tensor, " ... seq"] -> Float[Tensor, " ... seq d"]):
        return run_rope(self.dim, self.theta, self.context_length, x, pos_ids)
    def extra_repr(self):
        return f"context_length={self._freq_cis_cache.shape[0]}, dim/2={self._freq_cis_cache.shape[1]}"
```

2.3.2. Transformer Block

每个 Transformer 块由以下子模块组成：多头自注意力机制（Multi-head Self-Attention）、第一归一化层（RMSNorm）、前馈网络（Feed-Forward Network）、第二归一化层（RMSNorm）等，核心实现逻辑位于 adapters.py 的 run_transformer_block 函数，整体流程遵循残差连接模式。

（1）多头自注意力：实现于 run_multihead_self_attention_with_rope 函数，核心步骤包括：

①投影操作：通过线性变换将输入转换为查询（Q）、键（K）、值（V）。

```
q = run_linear(d_model, num_heads * d_head, q_proj_weight, x)
k = run_linear(d_model, num_heads * d_head, k_proj_weight, x)
v = run_linear(d_model, num_heads * d_head, v_proj_weight, x)
```

②RoPE 编码：对查询和键应用旋转位置编码。

```
q = run_rope(d_head, rope_theta, context_length, q, pos_ids)
k = run_rope(d_head, rope_theta, context_length, k, pos_ids)
```

③缩放点积注意力：在每个注意力头内计算注意力权重。

```
attn_output = run_scaled_dot_product_attention(q, k, v, mask)
```

④输出投影：将多头注意力结果拼接并线性变换。

```
output = run_linear(num_heads * d_head, d_model, o_proj_weight, attn_output)
```

(2) RMSNorm 归一化：此处用于替代传统 LayerNorm，计算根均方后缩放：

$output = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \times weight$ ，无需仿射变换，减少参数量，提升推理速度。

```
class RMSNorm(nn.Module):
    def __init__(self, hidden_size: int, eps: float = 1e-5, device=None):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size, device=device))
        self.eps = eps
    def forward(self, x):
        d_model = self.weight.shape[0]
        return run_rmsnorm(d_model, self.eps, self.weight, x)
```

(3) 前馈网络：采用 SwiGLU 激活函数的前馈网络，相比 ReLU 等激活函数，具有更好的表达能力和梯度特性，实现于 run_swiglu 函数。

```
def run_swiglu(d_model, d_ff, w1_weight, w2_weight, w3_weight, in_features):
    x1 = run_linear(d_model, d_ff, w1_weight, in_features)
    x2 = run_linear(d_model, d_ff, w3_weight, in_features)
    x = run_silu(x1) * x2
    return run_linear(d_ff, d_model, w2_weight, x)
```

(4) 输出层：模型最终输出通过以下步骤实现：最终归一化（对 Transformer 块的输出应用 RMSNorm）、线性变换（通过 lm_head 将模型维度映射到词汇表大小）。输出形状：(batch_size, sequence_length, vocab_size)，表示每个位置的未归一化概率分布。

```
x = run_rmsnorm(self.d_model, 1e-5, self.ln_final.weight, x)
logits = run_linear(self.d_model, self.vocab_size, self.lm_head.weight, x)
```

(5) 生成机制，类似于 LSTM，此处不在赘述，详见具体代码实现。

2.3. 模型训练 (train.py)

2.3.1. 可视化监控

```
class TrainingVisualizer:
    def update_training_loss(self, iteration, loss, lr):
        self.train_losses.append(float(loss))
        self.learning_rates.append(float(lr))
        self.iterations.append(int(iteration))

    def update_validation_loss(self, iteration, loss):
        self.val_losses.append(float(loss))
        self.val_iterations.append(int(iteration))

    def plot_training_progress(self, current_iteration):
```

```
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(15, 10))
```

实时监控训练过程，记录损失曲线和学习率变化

2.3.2 数据加载与预处理

```
def get_memmap_dataset(path, dtype=np.int32):
    arr = np.memmap(path, dtype=dtype, mode="r") # 内存映射，避免全量加载
    return arr

def get_batch(memmap_arr, batch_size, context_length):
    ix = np.random.randint(0, N - context_length - 1, size=(batch_size,))
    x = np.stack([memmap_arr[i:i+context_length] for i in ix])
    y = np.stack([memmap_arr[i+1:i+context_length+1] for i in ix])
    return torch.tensor(x, dtype=torch.long), torch.tensor(y, dtype=torch.long)
```

高效加载大规模训练数据，支持内存映射和批量生成

2.3.3. 模型与优化器初始化

```
with open(CONFIG_PATH, 'r') as f:
    config = json.load(f)
model = BasicsTransformerLM(**config["model"])
model, device = _to_device_and_compile(model)
AdamW = get_adamw_cls()
optimizer = AdamW(model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
```

配置模型架构、优化器参数和训练超参数

2.3.4. 断点

```
if args.resume_checkpoint:
    resume_ckpt_path = pathlib.Path(__file__).resolve().parent.parent /
    f"checkpoints_transformer/ckpt_iter{args.resume_checkpoint}.pt"
    if os.path.exists(resume_ckpt_path):
        start_iter = run_load_checkpoint(resume_ckpt_path, model, optimizer)
```

从检查点恢复训练，确保训练过程可中断

2.3.5. 训练循环

```
for iteration in pbar:
    model.train()
    x, y = get_batch(train_data, args.batch_size, args.context_length)
    x, y = x.to(device), y.to(device)
    logits = model(x)
    loss = run_cross_entropy(
        logits.reshape(-1, logits.shape[-1]),
        y.reshape(-1)
    )
    lr = run_get_lr_cosine_schedule(iteration, args.lr, args.min_lr, args.warmup_iters, args.cosine_iters)
    optimizer.zero_grad()
    loss.backward()
    run_gradient_clipping(model.parameters(), args.clip_grad_norm)
    optimizer.step()
```

执行前向传播、损失计算、反向传播和参数更新

学习率: $\text{lr}(t) = \text{min_lr} + \frac{1}{2}(\text{max_lr} - \text{min_lr}) \left(1 + \cos\left(\frac{t}{T}\pi\right)\right)$

交叉熵损失: $\mathcal{L} = -\sum_{i=1}^V y_i \log(p_i)$

梯度裁剪: $\text{clipped_grad} = \begin{cases} \nabla W \cdot \frac{\text{clip_norm}}{\|\nabla W\|} & \text{if } \|\nabla W\| > \text{clip_norm} \\ \nabla W & \text{otherwise} \end{cases}$

2.3.6. 验证与保存

```
if (iteration + 1) % args.val_interval == 0:
    model.eval()
    with torch.no_grad():
        val_losses = []
        for x_val, y_val in memmap_val_iterator(val_data, args.batch_size, args.context_length):
            val_logits = model(x_val)
            val_loss = run_cross_entropy(val_logits.reshape(-1, val_logits.shape[-1]), y_val.reshape(-1))
            val_losses.append(val_loss.item())
        val_loss_mean = np.mean(val_losses)
    if (iteration + 1) % args.save_interval == 0:
        ckpt_name = os.path.join(args.save_path, f'ckpt_iter{iteration+1}.pt')
        run_save_checkpoint(model, optimizer, iteration+1, ckpt_name)
        visualizer.plot_training_progress(iteration+1)
        visualizer.save_training_data()
```

验证计算: $\text{val_loss} = \frac{1}{N} \sum_{j=1}^N \mathcal{L}_j$

定期验证模型性能，保存检查点和训练状态

2.4. 模型推理生成脚本 (generate.py)

2.4.1 配置与初始化

```
parser.add_argument("--prompt", type=str, default="Once upon a time, there was a man", help="Input prompt for generation")

TOKENIZER_DIR = pathlib.Path(__file__).resolve().parent.parent / "tokenizer"
VOCAB_PATH = os.path.join(TOKENIZER_DIR, "tinystories_bpe_vocab.pkl")
```

定义命令行接口参数

```
with open(VOCAB_PATH, 'rb') as f:
    vocab = pickle.load(f)

tokenizer = BPETokenizer(vocab=vocab, merges=merges, special_tokens=special_tokens)
```

从 pickle 文件加载预训练的 BPE 词汇表和合并规则，初始化分词器

2.4.2. 模型加载与编译

```
with open(CONFIG_PATH, 'r') as f:
    config = json.load(f)

model = BasicsTransformerLM(**config["model"])

model, device = _to_device_and_compile(model)

model.eval()
```

```
with open(CKPT_PATH, 'rb') as f:
    checkpoint = torch.load(f, weights_only=False)
    model.load_state_dict(checkpoint['model_state_dict'])
```

2. 4. 3. 文本生成流程

输入预处理

```
input_ids = tokenizer.encode(args.prompt)
input_tensor = torch.tensor([input_ids], dtype=torch.long, device=device)
```

自回归解码

```
output_tokens = model.generate(
    input_tensor,
    max_new_tokens=args.max_new_tokens,
    temperature=args.temperature,
    top_k=args.top_k,
    eos_token_id=256
)
```

输出后处理

```
output_ids = output_tokens[0].cpu().numpy().tolist()
full_ids = input_ids + output_ids
text = tokenizer.decode(full_ids)
```

2. 4. 4. 增量解码

```
for _ in range(max_new_tokens):
    x = x[:, -self.context_length:]
    logits = self.forward(x)
    next_token_logits = logits[:, -1] / temperature
    if top_k:
        topk_values, _ = torch.topk(next_token_logits, k=top_k)
        threshold = topk_values[:, -1]
        mask = next_token_logits < threshold
        next_token_logits.masked_fill(mask, float("-inf"))
    next_token_id = torch.multinomial(run_softmax(next_token_logits), 1)
    if eos_token_id is not None and next_token_id.item() == eos_token_id:
        break
    x = torch.cat((x, next_token_id), dim=-1)
```

3. 代码运行

3. 1. 为确保复杂 Transformer 模型实现的可靠性，我们引入了 Stanford CS336 课程（2025 年春季）assignment 1 的测试代码，用于相关模块的功能测试。

批量测试

```
pytest
```

单个测试

```
pytest <python_file_path>
```

3. 2. 训练 Tokenizer。

```
python scripts/train_bpe.py
```

训练完成后，使用训练好的分词器，将原始 `txt` 格式的训练数据编码为整数 ID，并保存为一维数组的 `dat` 格式文件，以便在训练过程中快速读取。

```
python scripts/tokenize data.py
```

3.3. 训练 LSTM

```
python lstm scripts/train.py
```

训练结束后，测试其生成文本的能力

```
python lstm scripts/generate.py
```

3.4. 训练 Transformer

python transformer scripts/train.py

训练结束后，测试其生成文本的能力

python transformer scripts/generate.py

3.5. 通过调整 `lstm_scripts/config.json`、`transformer_scripts/config.json` 中的关键超参数，重复执行 3.3-3.4 节所述的流程，以探究不同参数配置对模型训练效果与资源消耗的影响。

七、实验结果

1.Transformers 模型相关模块的功能测试结果显示，所有测试均已通过。其中，有一个测试用例的状态为“xpassed”（预期失败但实际通过），这归因于我们实现的 BPE 算法内存效率优于测试基准的预期。在处理大规模数据时，算法未触发预设的内存超限警告，故标记为此状态。

```
(DM) lsx@escenter55-SYS-7049GP-TRT:~/code/DM-Group-Project/project2$ pytest
===== test session starts =====
platform linux -- Python 3.13.9, pytest-8.3.4, pluggy-1.6.0
rootdir: /home/lsx/code/DM-Group-Project/project2
plugins: jaxtyping-0.3.2
collected 48 items

tests/test_data.py . [ 2%]
tests/test_model.py ..... [ 29%]
tests/test_nn_utils.py ... [ 35%]
tests/test_optimizer.py .. [ 39%]
tests/test_serialization.py . [ 41%]
tests/test_tokenizer.py .....X [ 93%]
tests/test_train_bpe.py ... [100%]

===== 47 passed, 1 xpassed in 33.51s =====
```

2.Tokenizer 训练完成并已保存。我们随后执行了集成测试，验证了其编码、分词与解码功能，所有输出结果均正确，确认了 Tokenizer 的功能正确性。

```
(DM) lxx@escenter55-SYS-7049GP-TRT:~/code/DM-Group-Project/project2$ python scripts/train_bpe.py  
Training BPE: 100% [██████████] 9743/9743 [11:18:00:00, 14.35merge/s, vocab_size=1e+4, merges=9743, tokens=59933]  
longest token: b' accomplishment' length: 15
```

```
(DM) lxx@mesenter55-SYS-7049GP-TRT:~/code/DM-Group-Project/project2$ python scripts/tokenize_data.py
=== Test BPETokenizer ===

Original Text: Once upon a time, there was a little robot.
Encoding: [430, 439, 259, 398, 44, 401, 283, 259, 390, 2052, 46]
Tokenization: ['Once', ' upon', ' a', ' time', ',', ' there', ' was', ' a', ' little', ' robot', '.']
Decoding: Once upon a time, there was a little robot.
Fully Restored: True


Original Text: Hello world! <|endoftext|> Some more text.
Encoding: [1183, 1569, 33, 32, 256, 2770, 675, 799, 983, 46]
Tokenization: ['Hello', ' world', '!', ' ', ' ', '<|endoftext|>', ' Some', ' more', ' te', 'xt', '.']
Decoding: Hello world! <|endoftext|> Some more text.
Fully Restored: True


Original Text: <|endoftext|>
Encoding: [256]
Tokenization: ['<|endoftext|>']
Decoding: <|endoftext|>
Fully Restored: True


Original Text: Shengxiang Lin
Encoding: [83, 489, 103, 120, 105, 1649, 361, 270]
Tokenization: ['S', 'hen', 'g', 'x', 'i', 'ang', ' L', 'in']
Decoding: Shengxiang Lin
Fully Restored: True


Original Text: 你好，世界！
Encoding: [228, 189, 160, 229, 165, 189, 239, 188, 140, 228, 184, 150, 231, 149, 140, 239, 188, 129]
Tokenization: ['你', '好', '世', '界', '！', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']
Decoding: 你好，世界！
Fully Restored: True


Counting tokens: 100% | ██████████ 15600057/15600057 [57:28<00:00, 4523.83it/s]
Tokenizing: 100% | ██████████ 15600057/15600057 [59:14<00:00, 4388.52it/s]
Counting tokens: 100% | ██████████ 157832/157832 [00:34<00:00, 4509.49it/s]
Tokenizing: 100% | ██████████ 157832/157832 [00:35<00:00, 4438.86it/s]
```

3.LSTM 模型训练与文本生成实例

```
❖ (DM) lxx@escenter55-SYS-7049GP-TRT:~/code/DM-Group-Project/project2$ python lstm_scripts/train.py
Using device: cuda
Loading datasets...
Train data size: 540796778, Val data size: 5461210
Starting training from iteration 0 to 10000
Validation interval: every 100 iterations
Save interval: every 10000 iterations
Training: 1% | 99/10000 [00:10<12:08, 13.59it/s, loss=10.4952, ce_loss=9.1253, reg_loss=1.3698, lr=
iter 99: train_loss = 10.4952, VALID loss = 9.1152, lr = 3.96e-05
Training: 2% | 199/10000 [00:18<12:13, 13.36it/s, loss=6.9237, ce_loss=5.8889, reg_loss=1.0348, lr=
iter 199: train_loss = 6.9237, VALID loss = 5.7864, lr = 7.96e-05
Training: 3% | 299/10000 [00:25<12:04, 13.39it/s, loss=6.4339, ce_loss=5.8095, reg_loss=0.6244, lr=
iter 299: train_loss = 6.4339, VALID loss = 5.7649, lr = 1.20e-04
.....
```

```

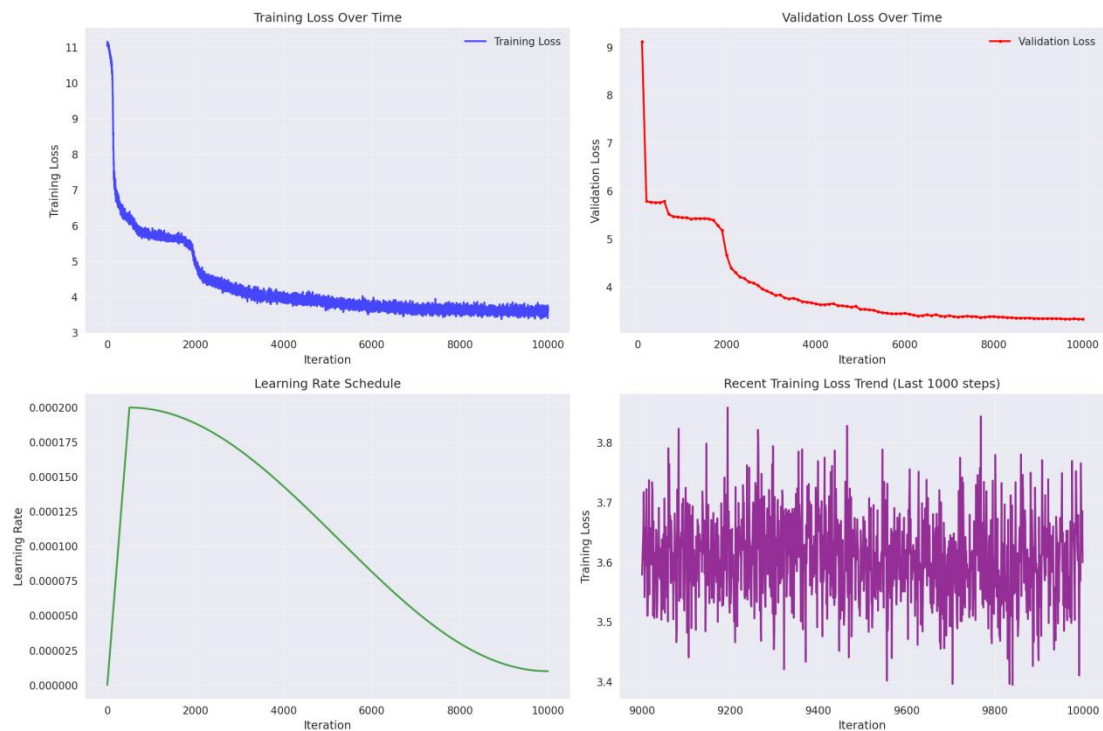
Training: 96%[ ] 9599/10000 [12:06<00:29, 13.66it/s, loss=3.5760, ce_loss=3.5184, reg_loss=0.0576, lr
iter 9599: train_loss = 3.5760, VALID loss = 3.3299, lr = 1.08e-05
Training: 97%[ ] 9699/10000 [12:14<00:21, 13.78it/s, loss=3.6454, ce_loss=3.5919, reg_loss=0.0534, lr
iter 9699: train_loss = 3.6454, VALID loss = 3.3295, lr = 1.05e-05
Training: 98%[ ] 9799/10000 [12:21<00:14, 13.67it/s, loss=3.6055, ce_loss=3.5612, reg_loss=0.0443, lr
iter 9799: train_loss = 3.6055, VALID loss = 3.3345, lr = 1.02e-05
Training: 99%[ ] 9899/10000 [12:29<00:07, 13.64it/s, loss=3.4360, ce_loss=3.3886, reg_loss=0.0474, lr
iter 9899: train_loss = 3.4360, VALID loss = 3.3266, lr = 1.01e-05
Training: 100%[ ] 9999/10000 [12:36<00:00, 13.56it/s, loss=3.5998, ce_loss=3.5403, reg_loss=0.0595, lr
iter 9999: train_loss = 3.5998, VALID loss = 3.3259, lr = 1.00e-05
Checkpoint saved to ./lstm_checkpoints/ckpt_iter10000.pt
Visualization saved to: ./lstm_checkpoints/visualizations/training_progress_iter_10000.png
Training data saved to: ./lstm_checkpoints/visualizations/training_data.json
Training: 100%[ ] 10000/10000 [12:39<00:00, 13.16it/s, loss=3.5998, ce_loss=3.5403, reg_loss=0.0595, l
Training completed! Generating final visualization...
Visualization saved to: ./lstm_checkpoints/visualizations/training_progress_iter_10000.png
Training data saved to: ./lstm_checkpoints/visualizations/training_data.json
Training data and visualizations saved to ./lstm_checkpoints/visualizations

```

```
(DM) lsx@escenter55-SYS-7049GP-TRT:~$ nvidia-smi
Sat Nov 1 15:27:48 2025
+-----+
| NVIDIA-SMI 570.172.08                Driver Version: 570.172.08      CUDA Version: 12.8   |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+-----+-----+-----+-----+
|  0   NVIDIA GeForce RTX 4090        Off | 00000000:18:00.0 Off |          94%      Off |
| 51%   67C   P2              236W / 450W | 3480MiB / 24564MiB |          94%      Default |
|                                           | N/A             |
+-----+-----+-----+-----+-----+-----+
|  1   NVIDIA GeForce RTX 4090        Off | 00000000:86:00.0 Off |          0%      Off |
| 30%   32C   P8              20W / 450W |  4MiB / 24564MiB |          0%      Default |
|                                           | N/A             |
+-----+-----+-----+-----+-----+-----+
|  2   NVIDIA GeForce RTX 4090        Off | 00000000:AF:00.0 Off |          0%      Off |
| 30%   33C   P8              16W / 450W |  4MiB / 24564MiB |          0%      Default |
|                                           | N/A             |
+-----+-----+-----+-----+-----+-----+

+-----+
| Processes:                                     GPU Memory |
|  GPU   GI    CI          PID    Type    Process name      Usage   |
|  ID    ID    ID             |          |
+-----+-----+-----+-----+-----+-----+
|    0   N/A  N/A         3228101    C     python              3470MiB |
+-----+-----+-----+-----+-----+-----+
```

```
(DM) lsx@escenter55-SYS-7049GP-TRT:~/code/DM-Group-Project/project2$ python lstm_scripts/generate.py
Input: Once upon a time, there was a man
Output: Once upon a time, there was a man named Lily. Tim had a new monster and it care, Lucy, and Emma
carried anything to berries.
From and
On what Tim said, "Yes, I want to help what the mouse has a little dog for warmer. Openers exerc yourse
lf with swimsuits into "Sure!" The bear learned something home.
Lucy lived again. Then, "Sara how behind Sam for helps, half, me with his big not wagon: Her bubble! Ca
n Tim stuck. "Lily that happen!" Sam and Lily was on and said, "HAnne with without sometimes she were m
any selfish fast. The olive arrived. Her friends will know her do." "
```



4. Transformer 模型训练与文本生成实例

```
(DM) lsx@escenter55-SYS-70496P-TRT:~/code/DM-Group-Project/project2$ python transformer_scripts/train.py
Using device: cuda
Loading datasets...
Train data size: 540796778, Val data size: 5461210
Starting training from iteration 0 to 10000
Validation interval: every 100 iterations
Save interval: every 10000 iterations
Training: 1%|          | 99/10000 [00:18<13:16, 12.43it/s, loss=7.0797, lr=3.96e-05]
iter 99: train_loss = 7.0797, VALID loss = 7.0111, lr = 3.96e-05
Training: 2%|          | 199/10000 [00:30<13:10, 12.40it/s, loss=4.8674, lr=7.96e-05]
iter 199: train_loss = 4.8674, VALID loss = 4.7927, lr = 7.96e-05
Training: 3%|          | 299/10000 [00:38<13:03, 12.38it/s, loss=3.9525, lr=1.20e-04]
iter 299: train_loss = 3.9525, VALID loss = 3.7937, lr = 1.20e-04

.....

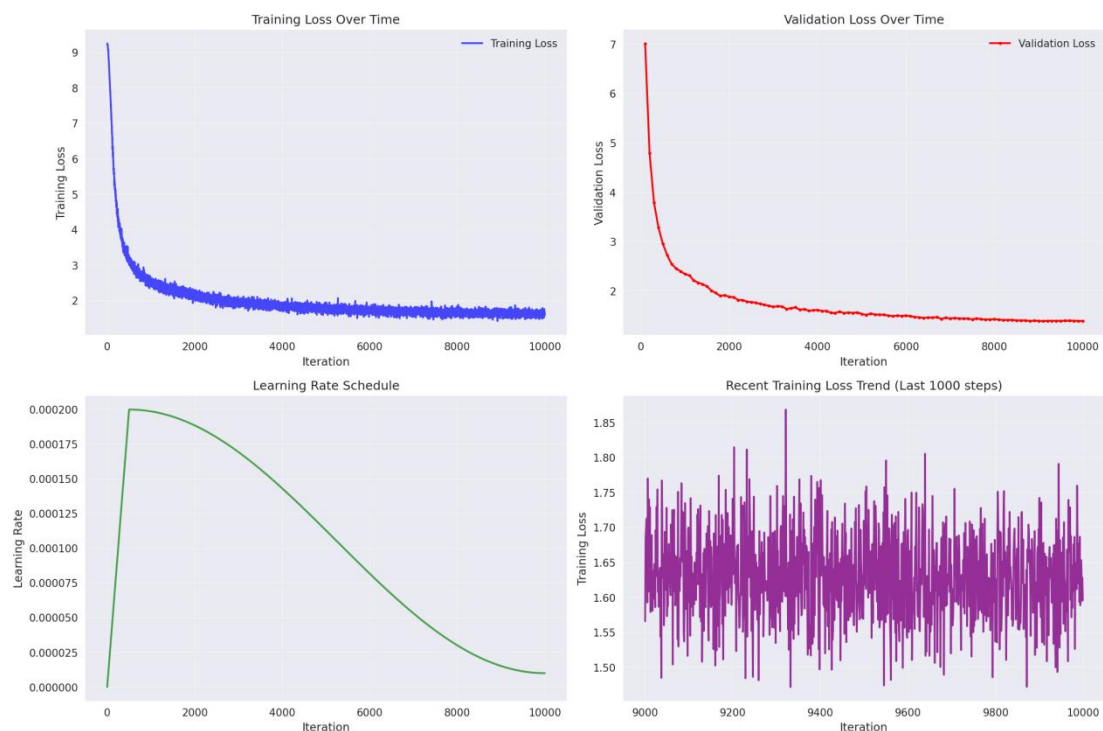
iter 9699: train_loss = 1.6467, VALID loss = 1.3980, lr = 1.05e-05
Training: 98%|          | 9799/10000 [14:03<00:16, 12.14it/s, loss=1.5371, lr=1.02e-05]
iter 9799: train_loss = 1.5371, VALID loss = 1.3975, lr = 1.02e-05
Training: 99%|          | 9899/10000 [14:11<00:08, 12.15it/s, loss=1.6848, lr=1.01e-05]
iter 9899: train_loss = 1.6848, VALID loss = 1.3940, lr = 1.01e-05
Training: 100%|         | 9999/10000 [14:20<00:00, 12.16it/s, loss=1.5959, lr=1.00e-05]
iter 9999: train_loss = 1.5959, VALID loss = 1.3950, lr = 1.00e-05
Checkpoint saved to ./transformer_checkpoints/ckpt_iter10000.pt
Visualization saved to: ./transformer_checkpoints/visualizations/training_progress_iter_10000.png
Training data saved to: ./transformer_checkpoints/visualizations/training_data.json
Training: 100%|         | 10000/10000 [14:23<00:00, 11.58it/s, loss=1.5959, lr=1.00e-05]
Training completed! Generating final visualization...
Visualization saved to: ./transformer_checkpoints/visualizations/training_progress_iter_10000.png
Training data saved to: ./transformer_checkpoints/visualizations/training_data.json
Training data and visualizations saved to ./transformer_checkpoints/visualizations
```

```
(DM) lsx@escenter55-SYS-7049GP-TRT:~$ nvidia-smi
Sat Nov 1 15:11:15 2025

+-----+
| NVIDIA-SMI 570.172.08                  Driver Version: 570.172.08      CUDA Version: 12.8   |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |
+-----+-----+-----+-----+-----+-----+
|  0   NVIDIA GeForce RTX 4090          Off | 00000000:18:00.0 Off |          Off        |
| 81%   76C    P2              405W /  450W | 7052MiB / 24564MiB |   94%      Default  |
|                                           | N/A            |
+-----+-----+-----+-----+-----+-----+
|  1   NVIDIA GeForce RTX 4090          Off | 00000000:86:00.0 Off |          Off        |
| 30%   29C    P8              20W /  450W |  4MiB / 24564MiB |    0%      Default  |
|                                           | N/A            |
+-----+-----+-----+-----+-----+-----+
|  2   NVIDIA GeForce RTX 4090          Off | 00000000:AF:00.0 Off |          Off        |
| 30%   31C    P8              16W /  450W |  4MiB / 24564MiB |    0%      Default  |
|                                           | N/A            |
+-----+-----+-----+-----+-----+-----+

+-----+
| Processes:                               |
|  GPU   GI    CI          PID    Type   Process name                  GPU Memory |
|          ID    ID                                   |          Usage   |
+-----+-----+-----+-----+-----+-----+
|    0   N/A   N/A         3203009      C      python                      7042MiB |
+-----+-----+-----+-----+-----+-----+

(DM) lsx@escenter55-SYS-7049GP-TRT:~/code/DM-Group-Project/project2$ python transformer_scripts/generate.py
Input: Once upon a time, there was a man
Output: Once upon a time, there was a man with a big mustache. He liked to wear a wide smile. One day,
he wanted a friend to play with. He put on his face and said, "Hi! What do you want to do?"
A little girl said, "I'm Lily. I'm going to introduce myself to my friend." The man looked at her and s
miled. He said, "Hi, little girl. My name is Tom." But first, he had home. He was deaf as he could fix
his home.
Suddenly, a big bird flew down and took Tom's pale hat. Tom was scared, but he gave the hat back to the
man. "Here you go, little girl," said Tom, as he took the hat inside the house.
Lily was very happy to see her brother again. She was glad that her chance to go back to her land. Tom
thanked her for sharing the smallest hat and playing with her. From that day on, Tom liked friends, and
they all played together in the big, colorful land.
```



5.不同参数下的训练效果和资源使用情况

5.1. LSTM 模型参数实验 (vocab_size=10000)

5.1.1.网络层数 (num_layers)

固定参数: context_length=256, d_model=512

num_layers	train_loss	Val_loss	Train_time	GPU Memory Usage
4	3.4693	3.2335	568	3092
8	3.5998	3.3259	759	3480
16	5.8167	5.8161	1147	4454
32	5.7374	5.7409	2181	6704

本实验中，随着层数增加，模型性能先基本保持后显著下降（层数 ≥ 16 时损失剧增），同时训练时间与显存消耗近乎线性增长。表明对于此任务，过深的LSTM会导致优化困难且效率低下，4层是此配置下的最佳选择。

5.1.2.模型维度(d_model)

固定参数: context_length=256, num_layers=4

d_model	train_loss	Val_loss	Train_time	GPU Memory Usage
128	3.9752	3.7126	223	2612
256	3.9573	3.6219	564	2726
512	3.4693	3.2335	568	3092
1024	3.4460	3.0744	682	4098

本实验中，增大模型维度能持续提升性能（损失降低），但收益逐渐减小。资源消耗随之增加，但当维度从256提升至512时，训练时间未显著增加，显示了一定的计算效率优化。d_model=1024在取得最佳性能的同时，资源开销可控。

5.1.3.上下文长度 (context_length)

固定参数: d_model=512, num_layers=4

context_length	train_loss	Val_loss	Train_time	GPU Memory Usage
----------------	------------	----------	------------	------------------

64	3.6107	3.3153	318	1592
256	3.4693	3.2335	568	3092
512	3.4914	3.1978	792	5196
1024	3.3150	3.2274	1435	8992

本实验中，更长的上下文长度总体上带来了更好的性能（损失更低），但增长至 512 后收益不再明显。然而，其对训练时间和显存占用的影响是巨大的，几乎呈线性增长。这表明需要在性能与成本间进行权衡。

5.2. Transformer 模型参数实验（vocab_size=10000）

5.2.1. 网络层数 (num_layers)

固定参数：context_length=256, d_model=512, num_heads=16, rope_theta=10000, d_ff=1024

Num_layers	train_loss	Val_loss	Train_time	GPU Memory Usage
4	1.6237	1.5532	498	4460
8	1.6039	1.4102	820	6478
16	1.5353	1.3538	1460	10854
32	1.4275	1.3154	2744	18956

本实验中，增加 Transformer 的层数能持续提升性能。但同样地，资源消耗急剧增加。此实验中，32 层模型取得了最佳验证损失，但其训练成本最高。

5.2.2. 注意力头数 (num_heads)

固定参数：context_length=256, d_model=512, num_layers=4, rope_theta=10000, d_ff=1024

num_heads	train_loss	Val_loss	Train_time	GPU Memory Usage
8	1.6768	1.5333	476	3952
16	1.6237	1.5532	498	4460
32	1.7554	1.5190	624	5996
64	1.6857	1.5381	818	7964

本实验中，注意力头数对性能的影响并非单调，16 头或 32 头在此任务上表现较好。头数增加会显著增加计算量（时间）和显存占用，表明需要精细调优而非一味增多。

5.2.3. 前馈网络维度 (d_ff)

固定参数：context_length=256, d_model=512, num_layers=4, rope_theta=10000, num_heads=16

d_ff	train_loss	Val_loss	Train_time	GPU Memory Usage
512	1.8429	1.6161	482	4304
1024	1.6237	1.5532	498	4460
2048	1.6411	1.4557	634	5116
4096	1.6249	1.3697	746	5836

本实验中，增大 d_ff 维度总体上能提升模型能力（尤其验证损失从 1024 到 4096 持续改善），但同样伴随资源消耗的增长。d_ff=4096 取得了最佳的验证性能。

5.2.4. 上下文长度 (context_length)

固定参数：d_model=512, num_layers=4, num_heads=16, rope_theta=10000, d_ff=1024

context_length	train_loss	Val_loss	Train_time	GPU Memory Usage
64	2.1149	1.8098	239	1586
128	1.8942	1.6020	273	2586
256	1.6237	1.5532	498	4460
512	1.6055	1.5559	1118	10334

本实验中，更长的上下文窗口能显著提升性能（损失降低），但训练时间和显存消耗会急剧增加，且性能增益在达到一定长度后（如 256 到 512）变得不明显。

5.2.5. RoPE 旋转基数 (rope_theta)

固定参数：context_length=256, d_model=512, num_layers=4, d_ff=1024, num_heads=16

rope_theta	train_loss	Val_loss	Train_time	GPU Memory Usage
1000	1.7648	1.5484	499	4460
5000	1.6866	1.5445	500	4460
10000	1.6237	1.5532	498	4460
50000	1.7547	1.5296	517	4460

本实验中，RoPE 位置编码的基数 rope_theta 对模型性能存在一定影响，但其影响并非单调，最佳值可能存在于一个区间内（在本实验中 50000 取得了最佳的验证损失）。一个非常重要的发现是，改变 rope_theta 参数值几乎不影响训练时间和 GPU 内存占用。这是因为该参数仅改变了位置编码的计算方式，而不会改变模型的结构或中间激活值的大小，其计算和存储开销是恒定且低微的。

5.2.6. 模型维度 (d_model)

固定参数：context_length=256, num_layers=4, num_heads=16, d_ff=1024, rope_theta=10000

d_model	train_loss	Val_loss	Train_time	GPU Memory Usage
128	2.2776	2.0476	319	3970
256	1.9799	1.6903	369	4198
512	1.6237	1.5532	498	4460
1024	1.5836	1.3853	825	5636

在本实验中，增大模型维度能持续且显著地提升模型性能（训练损失与验证损失同步下降）。然而，更大的 d_model 也带来了显著增加的训练时间和 GPU 内存消耗。从 d_model=512 提升到 1024，验证损失有明显改善，但需要付出训练时间显存占用的代价。

八、实验问题以及解决过程

1. 在 BPE 实现中，为克服传统暴力算法内存开销大、效率低下的问题，我们采用了三项核心优化：首先，通过维护一个 pre_tokens_cnt 频率字典（其键为预分词后的整数 ID 元组，值为对应频率），将合并操作从遍历原始序列的 $O(n)$ 复杂度转化为直接更新字典条目的 $O(1)$ 操作，大幅减少了内存占用；其次，引入优先级队列来维护候选符号对及其频率，将每次查找最高频符号对的时间复杂度从 $O(v)$ 优化至 $O(\log v)$ ；最后，针对特殊 Token（如 `<|endoftext|>`）构建字典树

(Trie)，确保其在编码阶段能被优先识别且保持完整，从而在提升分词鲁棒性的同时，避免了训练过程中的错误拆分。这些策略共同实现了 BPE 训练效率的显著提升与资源消耗的有效控制。

2. 鉴于 Transformer 架构中 RoPE、多头注意力等模块的实现细节复杂且易错，我们为确保模型组件的稳健性，采取了以下工程实践：引入 Stanford CS336 课程的标准化测试套件，对每个核心模块进行严格的单元测试与集成测试；并遵循自底向上的开发流程，即先独立实现并验证基础组件的正确性，再将它们逐步集成为完整模型，从而有效降低了整体调试的复杂度。

九、实验小结

本次实验成功完成了 LSTM 与 Transformer 文本生成模型的实现与系统性对比研究。在本实验中，我们实现的 Transformer 在相关数据集上的文本生成效果显著优于 LSTM。通过控制变量的参数实验，系统性地揭示了模型复杂度（如层数、维度）与性能、资源消耗之间的权衡关系。实验过程中，我们有效解决了训练中的技术挑战，并通过模块化测试确保了代码可靠性。本项目不仅为模型选型与调优提供了实证依据，也体现了团队成员在架构设计、算法实现与实验分析上的高效协作与明确分工。

十、人员及分工

成员	分工
林圣翔	负责项目统筹、核心代码架构设计，Transformer 算法的核心实现、模型训练与文本生成、实验报告撰写等。
易怀川	负责 LSTM 算法的实现、模型训练与文本生成、实验报告的撰写工作等
李彦锟	负责多项底层功能模块的开发与集成测试、模型调参、实验报告的最终整理与汇总等